READING SHEET

Maps, Real-Time & Data Formats

Module 3 · Frontend: React & Next.js · Day 47 · Week 10

PROGRAM	WORK TYPE	EST. TIME
CodeOps · Full Stack	**Revision + mini-project**	**2–3 hours**

> **Tip How to use this sheet**
> Three things a delivery application needs that a dashboard does not: a map showing where the order is, a connection that keeps the screen current, and formatting that shows ETB and Addis time correctly. Part 4 covers the formats data takes when it leaves your application. Each part is independent — read the one your capstone actually needs first.

Part 1 · Maps with Leaflet

A delivery application without a map is a form. Leaflet is small, free and open, needs no API key, and works anywhere — which matters when the alternative bills per thousand views.

LEAFLET	A COMMERCIAL MAP API
Free, open source	Billed per thousand loads
No API key to manage	A key, and a budget alarm
About 40 kB	Considerably more
OpenStreetMap tiles	Proprietary tiles
Good enough for delivery	Better routing and traffic

> **Warning It touches window on import**
> Leaflet assumes a browser the moment it loads, so importing it in anything that renders on the server crashes the build. That is the single thing everyone hits first — and the reason for the dynamic import below.

Loading it without server rendering

app/track/MapPanel.jsx

```
"use client";
import dynamic from "next/dynamic";

const DeliveryMap = dynamic(() => import("./DeliveryMap"), {
  ssr: false,                    // the important line
  loading: () => <div className="h-80 bg-slate-100" /> });
```

You met dynamic import on Day 34 as a way to shrink a bundle. Here `ssr: false` is not an optimisation — it is the only way the library can load at all. Give the placeholder exactly the map's height, or the page jumps when it arrives.

The map itself

app/track/DeliveryMap.jsx

```
import { MapContainer, TileLayer, Marker, Popup }
  from "react-leaflet";
import "leaflet/dist/leaflet.css";

const BOLE = [8.9950, 38.7896];   // [lat, lng]

<MapContainer center={BOLE} zoom={14}
              style={{ height: "320px" }}>
  <TileLayer url="https://{s}.tile.openstreetmap.org/..."
             attribution="© OpenStreetMap contributors" />
  <Marker position={BOLE}>
    <Popup>Almaz · Bole · arriving 18:40</Popup></Marker>
</MapContainer>
```

Tiles, markers and popups are children, exactly as with a chart — adding a courier is adding another Marker. Three things people forget: **import the stylesheet** or the map renders as broken tiles, give the container an **explicit height**, and keep the **attribution** — it is a licence condition, not decoration.

Making it useful

NEED	HOW
Several couriers	Map over the array into Markers
A delivery zone	Polygon, or Circle with a radius
Fit everything on screen	Compute bounds, then fitBounds
A custom pin	L.icon with your own image
Not a map user	Also give the address as text

> **Warning Latitude first — and a map is not accessible**
> Leaflet takes [lat, lng]; GeoJSON uses [lng, lat]. Reversing them puts Bole somewhere in the Indian Ocean, and the mistake is easy to miss. Separately, tiles and pins mean nothing to a screen reader: always give the address, the area and the estimated time in words nearby. Note too that the public OpenStreetMap tile server is fine for coursework and not for production traffic.

Part 2 · Real-Time Data

Three ways to keep a screen current, in increasing order of cost and capability. Most applications need the simplest one.

	POLLING	SERVER-SENT EVENTS	WEBSOCKETS
Direction	Client asks	Server pushes	Both directions
Protocol	Ordinary HTTP	Ordinary HTTP	A separate protocol
Reconnects	Every request is new	Automatically	You write it
Right for	Order status	Live feeds	Chat, collaboration

Start at the left. Polling with the Day 41 refetch interval is already running in your project and costs nothing to maintain. Move to server-sent events when updates are frequent and one-directional;

move to WebSockets only when the client genuinely needs to send as well as receive. Choosing the most capable option first is the usual mistake.

Server-sent events

app/api/orders/[id]/stream/route.js

```
export async function GET(request, { params }) {
  const stream = new ReadableStream({
    async start(controller) {
      const send = (data) => controller.enqueue(
        `data: ${JSON.stringify(data)}\n\n`);
      send(await getStatus(params.id));
    } });

  return new Response(stream, { headers: {
    "Content-Type": "text/event-stream" } });
}
```

It is the Day 39 route handler returning a stream instead of a response — ordinary HTTP, so it passes through proxies and needs no new infrastructure. The message format is strict: `"data: "` then the payload then **two** newlines. One newline and the browser waits forever for a message that never completes.

Receiving it

a client component, with cleanup

```
"use client";
useEffect(() => {
  const es = new EventSource(`/api/orders/${id}/stream`);
  es.onmessage = (e) => setStatus(JSON.parse(e.data));
  es.onerror   = () => es.close();

  return () => es.close();     // Day 29, still essential
}, [id]);
```

> **Warning The cleanup is not optional**
> An unclosed EventSource survives navigation — ten visits to the tracking page means ten open connections and a server wondering why it is busy. Seed the initial value from the server render so the stream only updates it: nobody should watch a spinner for a value you already had.

Living with an open connection

REALITY	WHAT TO DO
The connection drops	SSE reconnects; show the state
The phone is on 3G	Assume gaps, resync on reconnect
The order is delivered	Close the stream deliberately
Nothing arrives at all	Fall back to polling

Show the connection state. A screen that quietly stops updating looks identical to one where nothing has happened, and a small "live" or "reconnecting" indicator is the difference between trust and confusion. Note that serverless hosting will not hold a socket open, so WebSockets normally mean a separate service — a Module 4 conversation.

Part 3 · Formatting for Ethiopia

Every application shows dates, times, numbers and money. The browser formats all of them correctly for a locale with no dependency — and doing it by hand is how "1200.5 birr" ends up on a receipt.

lib/format.js — build each formatter once

```
export const etb = new Intl.NumberFormat("en-ET", {
  style: "currency", currency: "ETB",
  maximumFractionDigits: 0 });

export const when = new Intl.DateTimeFormat("en-ET", {
  dateStyle: "medium", timeStyle: "short",
  timeZone: "Africa/Addis_Ababa" });   // EAT, UTC+3

etb.format(14200)                    // ETB 14,200
when.format(new Date(order.placedAt)) // 14 Jul 2026, 18:40
```

Creating a formatter is expensive; calling one is not, so export a single instance rather than constructing one inside a map over four thousand rows. Store money as an **integer** — santim, or whole birr — because floating point turns 0.1 + 0.2 into something no accountant will accept. And store dates in **UTC**, converting at the edge: a timestamp stored in local time is unrecoverable the moment anything moves.

The rest of Intl

relative times, percentages, lists

```
new Intl.RelativeTimeFormat("en", { numeric: "auto" })
  .format(-12, "minute");                 // 12 minutes ago

new Intl.NumberFormat("en-ET",
  { style: "percent" }).format(0.184);     // 18%

new Intl.ListFormat("en").format(["Doro Wat", "Tibs"]);
```

All of it is built into the browser and into Node, so installing a date library for formatting alone adds weight for something you already have. Where a library still earns its place is *arithmetic* on dates — adding months, comparing ranges, parsing odd input.

> **Tip The Ethiopian calendar**
> Ethiopia uses its own calendar and a 12-hour clock counted from dawn. If your users expect those, you need a dedicated library — Intl will not do it. Decide deliberately rather than by omission. On the other hand EAT is UTC+3 all year with no daylight saving, which removes a class of bug other regions live with.

Always name both the locale and the time zone. Relying on the environment means the server and the phone disagree and React reports a hydration mismatch — which, with a date, is almost always an unnamed time zone.

Part 4 · Data Formats & Files

FORMAT	USE IT FOR	WATCH OUT
JSON	APIs, configuration	No Date type — strings only
CSV	Export to a spreadsheet	Commas and quotes in values

FORMAT	USE IT FOR	WATCH OUT
Blob	Building a file to download	Revoke the URL afterwards
Base64	Embedding small data	A third larger than the bytes

`JSON.stringify` turns a Date into an ISO string and `JSON.parse` leaves it as a string. Nothing warns you — the value arrives as text and breaks the first time something compares it. Convert deliberately at the boundary, on both sides.

Exporting to CSV

```
                                                          the escaping is the whole problem
const cell = (v) => {
  const s = String(v ?? "");
  return /[",\n]/.test(s)
    ? `"${s.replace(/"/g, '""')}"`   // quote, and double
    : s;
};
```

> **Warning A restaurant called "Bole, Kitfo"**
> One comma in one value shifts every column after it. A naive join(",") works on your test data and corrupts the first real export — quote anything containing a comma, quote or newline. For reading CSV, use Papa Parse: handling quoted newlines and encodings by hand is a week you do not have.

Downloading a file

```
                                                                  a Blob, a link, a click
const blob = new Blob([text], { type: "text/csv" });
const url  = URL.createObjectURL(blob);
const a = document.createElement("a");
a.href = url; a.download = filename; a.click();
URL.revokeObjectURL(url);   // release the memory
```

The file is built and handed over entirely in the browser, so for data already on screen that is instant and costs nothing on the server. Past a few thousand rows, stream it from a route handler instead — building a hundred-thousand-row CSV in the browser will freeze the tab.

Files coming in

```
                                                          a File is a Blob with a name
const file = event.target.files[0];
file.name; file.size; file.type;

const text = await file.text();   // or arrayBuffer()
```

Check the **size** before reading — a 200 MB file will hang the tab — and never trust `file.type`, which comes from the browser and can say anything. Validate on the server before you store or process it: the Day 42 principle, applied to uploads.

Assigned Reading

Required

1. **MDN — Using server-sent events** What / why: EventSource, the message format, and how reconnection works.

2. **MDN — Intl** What / why: NumberFormat, DateTimeFormat and RelativeTimeFormat with every option.

Optional — go deeper

- **React Leaflet** — components, the container, and the setup steps people miss.
- **OpenStreetMap tile usage policy** — what the free tile server permits, before you deploy anything real.

Exercises

Test the CSV export with a value containing a comma and one containing a quotation mark.

1. Build the map as a dynamic client component with ssr:false and a sized placeholder.
2. Place a courier and a delivery marker, fit the view to both, and repeat it in text.
3. Add a stream route handler for the order status with correct SSE framing.
4. Consume it with EventSource, seeded from the server, closing it on unmount.
5. Show a connection indicator, and close the stream deliberately on delivery.
6. Write lib/format.js with etb, when and relative, and use it everywhere.
7. Add a CSV export button, then test it with awkward values.

Mini-Project · Tracking a Delivery

> **Tip What you build**
> A tracking page with a Leaflet map showing the courier and the delivery address, a live status over server-sent events seeded from the server render, every value formatted through one shared module, and a CSV export of the order history that survives awkward data.

Requirements

- The map loaded with ssr:false, stylesheet imported, explicit height, attribution kept.
- The same delivery information available as text beside the map.
- A stream route handler using "data: " and two newlines.
- An EventSource seeded from server data, closed on unmount, with a visible state.
- lib/format.js exporting etb, when and relative, each built once.
- CSV export that correctly quotes commas, quotes and newlines.

Check yourself

- Does the build succeed, and does the page avoid jumping when the map appears?
- Navigate away from the tracking page five times — how many connections are open?
- Is every price and time going through the shared format module?
- Does the CSV survive a value containing a comma and a quotation mark?
- Can someone who cannot see the map still find where their order is?